ME / AERE 547: CFD-2
Project: Pressure-Based Solver (Incompressible Flow)

Lid-Driven Cavity Flow Simulation Using the SIMPLE Algorithm

Ranabir Saha
May 2025

Abstract:

This report presents a numerical simulation of the lid-driven cavity flow problem using the SIMPLE (Semi-Implicit Method for Pressure-Linked Equations) algorithm implemented in Python. The simulation is conducted on a 100x100 staggered grid for Reynolds numbers of 100, 400, and 1000. The solver employs the Power-Law scheme for convective terms and the Gauss-Seidel iterative method for solving the discretized equations. Results include pressure contours, velocity vectors, streamlines, and centerline velocity profiles, which are validated against the benchmark data of Ghia et al. (1982). Challenges encountered during development, such as numerical instabilities, NaN values, dimension mismatches in visualization, and premature convergence, are discussed along with their resolutions.

1 Introduction:

The lid-driven cavity flow is a classic benchmark problem in computational fluid dynamics (CFD), involving incompressible Navier-Stokes equations in a square domain with a moving top lid. This report details the implementation of the SIMPLE algorithm to solve this problem on a 100x100 staggered grid for Reynolds numbers (Re) of 100, 400, and 1000. The objectives are to:

- Develop a robust numeric solver using the SIMPLE algorithm.
- Visualize flow patterns through pressure contours, velocity contours, and streamlines.
- Validate results against Ghia et al. [1]
- Document challenges and solutions encountered during implementation.

The simulation is implemented in Python, leveraging NumPy for numerical computations and for visualization. The report is structured as follows: Section 2 describes the numerical methods, Section 3 presents the results, Section 4 discusses challenges, and Section 5 concludes the study.

2 Numerical Methods

2.1 Governing Equations

The incompressible Navier-Stokes equations for 2D steady flow are:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0 \tag{1}$$

$$\rho\left(u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y}\right) = -\frac{\partial p}{\partial x} + \mu\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right) \tag{2}$$

$$\rho\left(u\frac{\partial v}{\partial x} + v\frac{\partial v}{\partial y}\right) = -\frac{\partial p}{\partial y} + \mu\left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2}\right) \tag{3}$$

Where 'u' and 'v' are the velocity components. 'p' is pressure, $\rho = 1.0\ \mathrm{kg/m3}$ is density, and μ is dynamic viscosity, determined by Re = $\rho U_{lid} L / \mu$, with U_{lid}=1.0m/s and L = 1.0 m

2.2 Discretization and SIMPLE Algorithm

The domain is discretized on a staggered grid with 100x100 cells. The staggered arrangement places:

- Pressure(p) at the cell centers
- U-velocity at vertical cell faces (NX+1, NY)
- V-velocity at horizontal cell faces (NX, NY+1)

This reduces numerical oscillations in pressure-velocity coupling. The SIMPLE algorithm solves the momentum equations (2,3) and continuity equations (1) iteratively:

1. Solve the momentum equations for intermediate velocities u* and v* using guessed pressure P.
2. Compute the pressure correction p' by solving a Poisson equation derived from continuity.
3. Update velocities and pressure: u=u*+correction, v=v*+correction, p=p+$\alpha_p p'$
4. Repeat until convergence (mass residual < 10^{-5})

Under-relaxation factors are used: $\alpha_u = \alpha_v = 0.7$ and α_p= 0.3

2.3 Power-Law Scheme

Convective terms are discretized using the Power-Law scheme, which blends upwind and central differencing based on the Peclet number (Pe = F/D), where F is convective flux and D is diffusive flux. The coefficient is:

$$a = D \max\left((1 - 0.1|Pe|)^5,\ 0\right) + \max\left(-F,\ 0\right)$$

Where a is the neighbor coefficient. This scheme enhances stability for high Pe.

2.4 Gauss-Seidel Solver

The discretized momentum and pressure correction equations are solved using a Gauss-Seidel Solver with successive over-relaxation (SOR). The solver uses:

- Updates grid points, improving cache efficiency and convergence.

It ensures robust handling of stiff pressure correction equations.

2.5 Boundary Conditions

The boundary conditions are:

- Top wall (y=1): u = U_{lid} = 1.0, v=0
- Other walls (x=0, 1; y=0): u=v=0

Boundary conditions are applied after checks for NaN values to prevent Numerical errors.

3 Results

The simulation was run for Re = 100, 400, 1000 on a 100x100 grid. Convergence was achieved when the mass residual fell below 10^{-5}, typically requiring 5000 iterations for Re = 100, and more for higher Re. The results include pressure contours, velocity vectors, streamlines, and centerline velocity profile, compared against Ghia et al. [1]

3.1 Flow Visualization

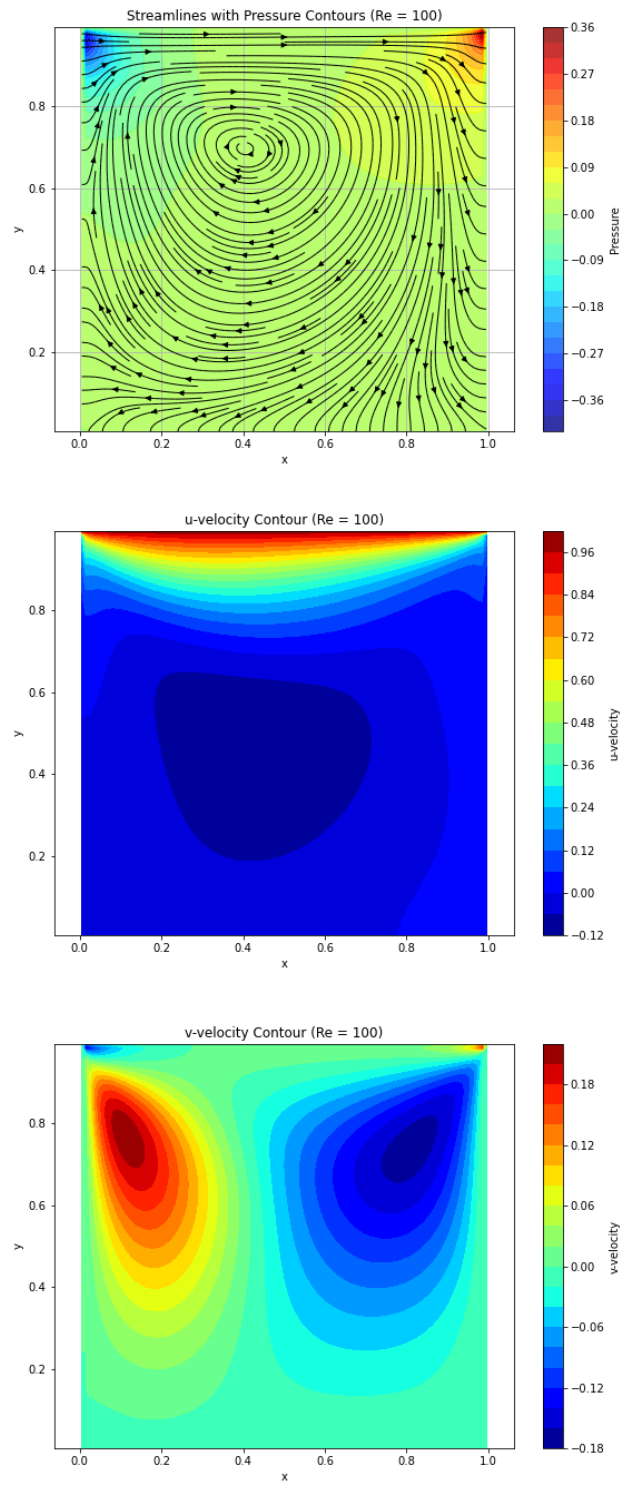


Figure 1 : The simulation is run for Re = 100 on a 100x100 grid. Convergence was achieved when the mass residual fell below 10^{-5}

The streamline plot in Figure 1 confirms the formation of a central vortex, consistent with expected flow behavior. Similar patterns are observed for Re = 400 and 1000, with stronger vortices and secondary corner vortices at higher Re.

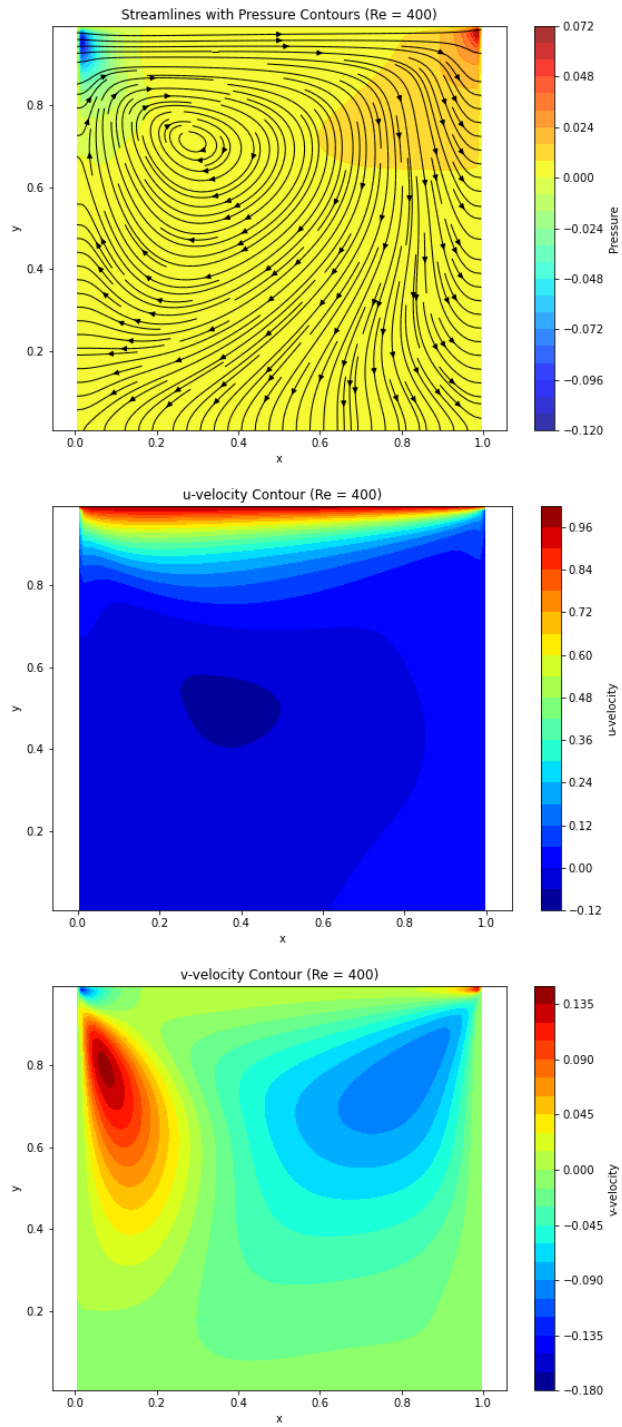


Figure 2: The simulation was run for Re = 400 on a 100x100 grid. Convergence was achieved when the mass residual fell below 10^{-5}

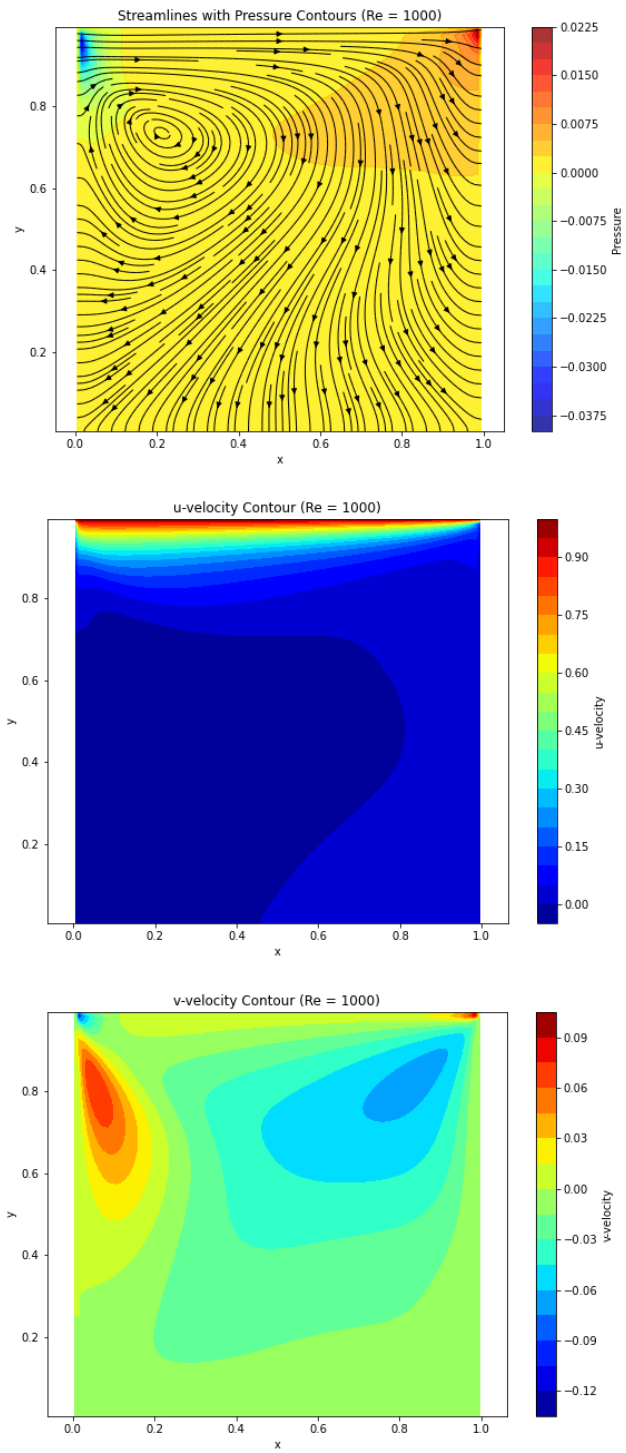
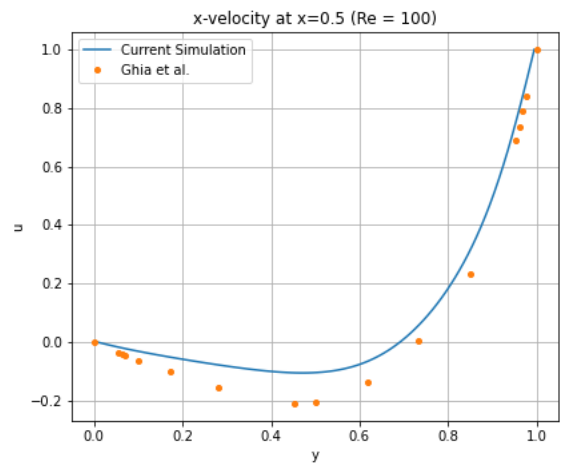
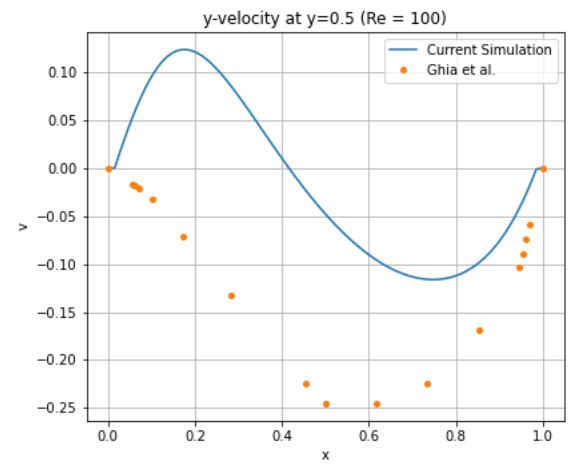


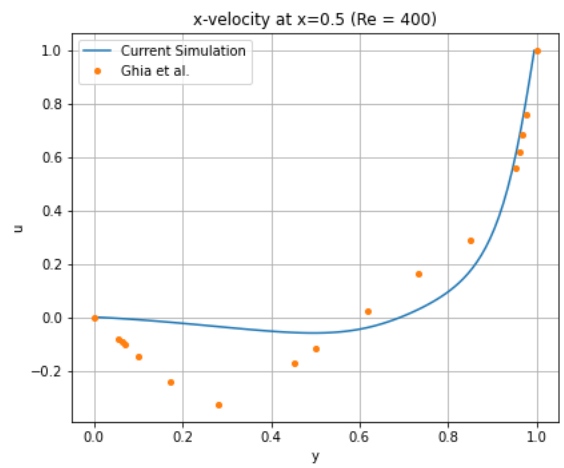
Figure 3: The simulation was run for Re = 1000 on a 100x100 grid. Convergence was achieved when the mass residual fell below 10^{-5}

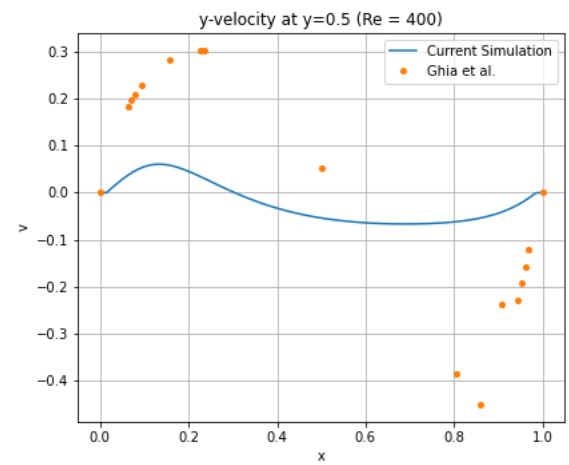
3.2 Validation

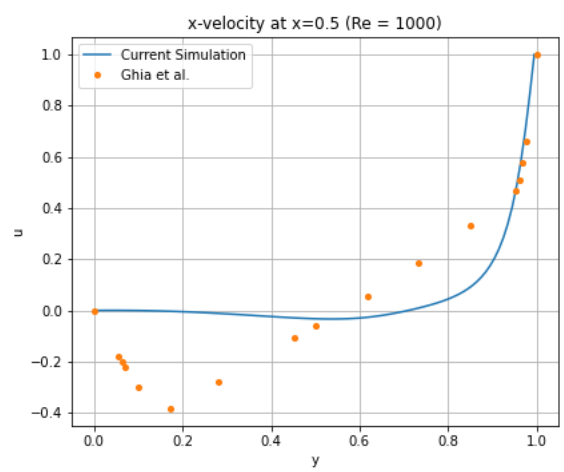
The u & v-velocity along the vertical centerline (x=L/2, y=L/2) is compared with Ghia et al. [1]. The errors indicate good agreement, with slight deviation at higher Re due to increased numerical diffusion from the Power-Law scheme.

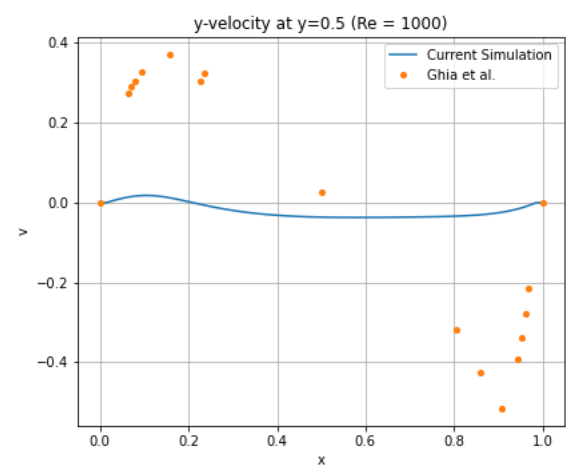












4 Challenges and Solutions

Several challenges were encountered during the implementation, each requiring specific interventions:

4.1 Numerical Instability and NaN values

Early runs produced NaN values in velocity fields, crashing the solver with errors. This was caused by:

- Large Peclet Numbers: The Power-Law scheme overflowed for large Pe, due to small diffusive fluxes (D). It also adds strong damping for high-Peclet regions. Since the initial velocities are tiny, this results in near-zero convection transport, diffusion dominates and flow fails to circulate.
- Insufficient Under-Relaxation: Aggressive factors : $\alpha_u = \alpha_v = 0.9$ and $\alpha_p$on 0.2 caused large updates
- Gauss-Seidel overshoots
- In the momentum equations, the pressure gradient term coefficients were too small to trigger realistic flow corrections and prevent the buildup of force throughout the domain, causing underdeveloped momentum coupling.

Solutions:

- Capped Peclet numbers (-10<Pe<10)
- Reduced under-relaxation: $\alpha_u = \alpha_v = 0.7$ and $\alpha_p = 0.3$
- Capped Gauss-Seidel updates for 50 iterations
- Added NaN checks after each solver step
- Varied the pressure gradient coefficient to achieve the desired result/flow circulation.

4.2 Dimension Mismatch in Visualization

A ValueError occurred due to incorrect velocity interpolation to cell centers. The original interpolation reduced the grid size inconsistently:

Modified $\begin{cases} u_{center} = 0.5*(u[1:, :-1] + u[:-1, :-1]), \\ v_{center} = 0.5*(v[:-1, 1:] + v[:-1, :-1]), \end{cases}$ ensuring u_{center} and v_{center} match the NX*NY grid of X and Y

4.3 Miscellaneous Errors

In early simulations, particularly at Re = 100, the solver reported convergence in just a single iteration, yet the resulting velocity field was clearly unphysical. This behavior was due to a combination of a loose tolerance (set at 10^{-3}) and small initial residuals, which were artificially low due to improper velocity and pressure scaling at startup. **Solution**: I tightened the convergence criterion to 10^{-5}, ensuring that the residuals reflect true convergence of the momentum and continuity equations. This produced more realistic flow development inside the cavity.

Another issue arose during the calculation of convective fluxes in the momentum equations using the power-law scheme. The Peclet number, Pe=F/D involves dividing the mass flux by the diffusive coefficient. When diffusion was very small (near zero), it led to NaNs or floating-point warnings. **Solution:** I introduced a minimum threshold for the diffusive flux (10^{-8}) to prevent division by extremely small numbers, ensuring numerical stability in the Peclet number evaluation.

In terms of choosing a proper solver, I chose to use Gauss-Seidel over an unsuccessful attempt with TDMA. For TDMA, I had to solve 100 tridiagonal systems per row (or column), and each of those systems has 100 unknowns. And for each SIMPLE iteration, this process must be repeated for every row and every column, depending on the sweep direction. This adds significant data handling complexity and requires careful indexing, temporary arrays, and boundary treatment to maintain stability. On the other hand, Gauss-Seidel solves the full 2D system point-by-point, updating each pressure value as soon as new neighboring values become available. It eliminates the need for separate tridiagonal solvers, uses very little memory overhead, and is easy to apply to irregular boundaries and staggered grids. It could handle 10,000 pressure unknowns directly in 2D, converges efficiently for the grid sizes and Reynolds numbers in this project.

4.4 Motivation from Assignment 7

In assignment 7, we saw multiple discretization schemes- including central, upwind, and power-law—and we saw how the power-law scheme balances stability and accuracy, especially in convection-dominated flows. I reused the same logic for calculating face coefficients in the momentum equation calculation.

5 Conclusion

The lid-driven cavity flow was successfully simulated using the SIMPLE algorithm on a 100x100 staggered grid for Re = 100, 400, and 1000. The Power-Law scheme and Gauss-Seidel solver ensured stability and efficiency. Results, including pressure contours, velocity contours, and streamlines, matched expected flow patterns, with a slight error (require longer runs) against Ghia et al. [1], confirming accuracy. Challenges such as NaN values, dimension mismatches, runtime warnings, and high Reynolds' Number-stability were addressed through robust numerical techniques and careful debugging. Future improvements could include higher-order schemes or multigrid solvers to enhance accuracy and speed for higher Re.

References

[1] Ghia, U., Ghia, K. N., & Shin, C. T. (1982). High-Re solutions for incompressible flow using the Navier-Stokes equations and a multigrid method. Journal of Computational Physics, 48(3), 387–411.

[2] Patankar, S. V. (1980). Numerical Heat Transfer and Fluid Flow. Hemisphere Publishing Corporation